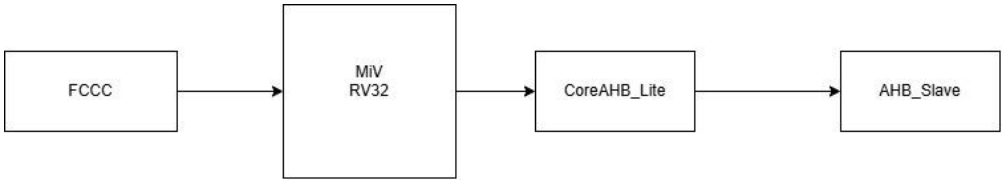
MI-V RV32 FIRMWARE INTEGRATION

sarathkc@krisemidesigntech.com

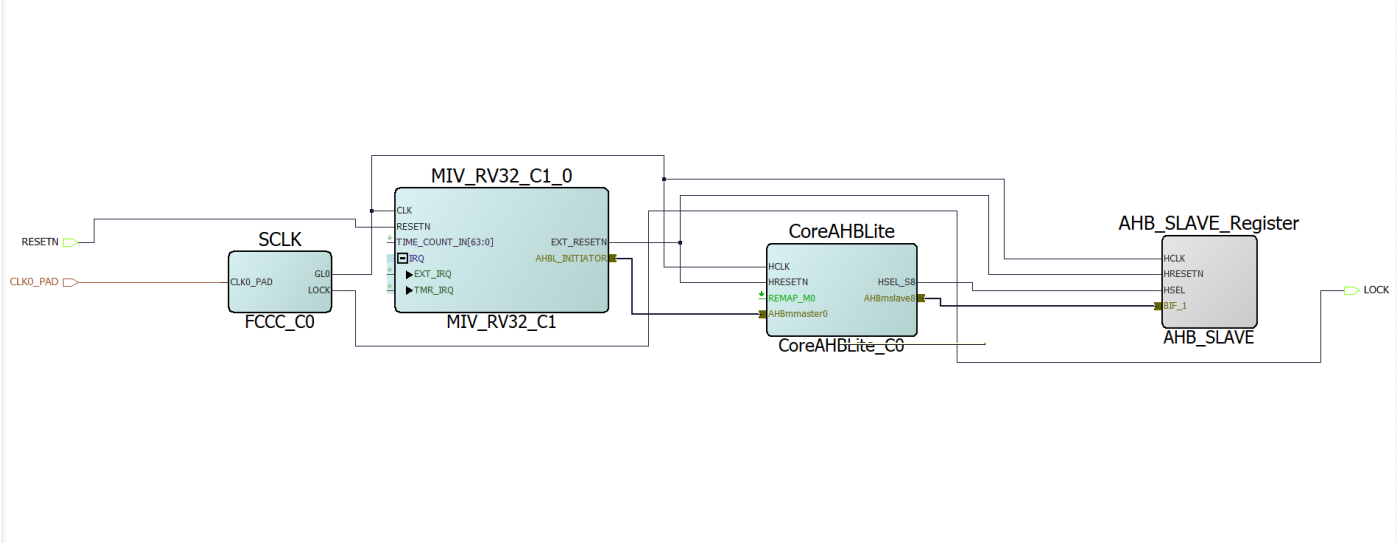
1. Project Objective

The objective was to replace the original SmartFusion2 MSS-based processor architecture with the **Microchip Mi-V RV32 RISC-V soft processor** and verify firmware execution through simulation.

1.1 Block Diagram



1.2 Smart Design View:



2. Mi-V RV32 Configuration

The original MSS was removed from the SmartDesign.

The current SmartDesign contains:

- MIV_RV32_C1
- CoreAHBLite_C0
- FCCC_C0
- AHB_SLAVE

The Mi-V configuration used:

Parameter	**Configuration**
Processor	Mi-V RV32
Version	3.1.200

Parameter	Configuration
ISA	RV32IMC
M extension	Enabled
C extension	Enabled
F extension	Disabled
Multiplier	Fabric
AHB Initiator	AHBLite
APB Initiator	Disabled
AXI Initiator	Disabled
ICache	Disabled
TCM	Enabled
BootROM	Disabled
TCM Access Support	Disabled
Debug	Disabled
Trace	Disabled
Hart ID	0

3. Memory Architecture

Two separate address regions are used.

3.1 Mi-V TCM

The processor TCM is mapped to: 0x4000_0000 – 0x4000_3FFF

This provides:16 KB TCM

The reset vector was therefore configured as: 0x4000_0000

This allows the processor to begin execution directly from TCM.

3.2 AHB Peripheral Space

The Mi-V AHB initiator is configured for: 0x8000_0000 – 0x8FFF_FFFF

The CoreAHBLite interconnect uses **slot 8** for the custom AHB slave.

Therefore: AHB Slave Base Address = 0x8000_0000

The firmware accesses the peripheral using:

#define AHB_SLAVE_REG (*(volatile uint32_t *)0x80000000u)

4. Firmware Development

A dedicated SoftConsole RISC-V project named:

test_riscv

was created.

The firmware uses a custom linker script because the program must execute from Mi-V TCM.

The linker memory definition is:

```
MEMORY
{
  TCM (rwx) : ORIGIN = 0x40000000, LENGTH = 16K
}
```

The reset entry point is:

ENTRY(_start)

- The startup code initializes the stack and calls main().

The intended application is:

```
#include <stdint.h>
#define AHB_SLAVE_REG (*(volatile uint32_t *)0x80000000u)
int main(void)
{
AHB_SLAVE_REG = 0x12345678u;
while (1)
  {
  }
}
```

5. HEX Address Offset Handling

The firmware HEX file originally contains the TCM's physical/linker address.

The SoftConsole build was configured with: “--change-section-lma *-0x40000000”

This removes the 0x40000000 address offset from the generated HEX data so that it can be loaded into the simulation RAM starting at memory index 0.

6. Mi-V TCM Simulation Loading

The generated TCM hierarchy was identified in QuestaSim:

DUT
└─ MIV_RV32_C1_0
  └─ MIV_RV32_C1_0
    └─ u_ipcore_0
      └─ gen_tcm0
        └─ u_subsys_TCM_0
          └─ tcm_ram
            └─ u_ram_0
              └─ gen_tcm
                └─ mem

The actual RAM array is: gen_tcm.mem

The simulation testbench uses:

```
$readmemh(
    "C:/MiVRV321/test_riscv/Release/test_riscv.mem",
    DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen_tcm.mem
);
```

This successfully loaded the firmware.

7. Simulation

Test bench :

```
`timescale 1ns / 1ps


module testbench;



  reg  CLK0_PAD;
  reg  RESETN;
  wire LOCK;

  initial begin
    CLK0_PAD = 1'b0;

    forever begin
      #5;
      CLK0_PAD = ~CLK0_PAD;
    end
  end

initial begin
  RESETN = 1'b0;

  $display("==============================================");
  $display(" RESET ASSERTED");
  $display("==============================================");
  $display("[%0t ns] RESETN = 0", $time);

  // Wait for CCC-generated Mi-V clock to become active
  wait (LOCK == 1'b1);
```

```
    // Give the generated clock some cycles before releasing reset
    #100;

    RESETN = 1'b1;

    $display("=============================================");
    $display(" RESET RELEASED AFTER FCCC LOCK");
    $display("=============================================");
    $display("[%0t ns] RESETN = 1", $time);
end



    frimware DUT (
        .CLK0_PAD (CLK0_PAD),
        .RESETN   (RESETN),
        .LOCK     (LOCK)
    );



    initial begin

        $display("=============================================");
        $display(" Loading Mi-V RV32 firmware into TCM");
        $display("=============================================");

        $readmemh(
            "C:/MiVRV321/test_riscv/Release/test_riscv.mem",
```

```
DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen_tcm.mem
    );

    $display("Firmware loaded into Mi-V TCM");
    $display("");

    // Print first 8 TCM locations
    $display("TCM[0] = %08h",
DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen_tcm.mem[0]);

    $display("TCM[1] = %08h",
DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen_tcm.mem[1]);

    $display("TCM[2] = %08h",
DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen_tcm.mem[2]);

    $display("TCM[3] = %08h",
DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen_tcm.mem[3]);

    $display("TCM[4] = %08h",
DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen_tcm.mem[4]);

    $display("TCM[5] = %08h",
```

```
DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen
_tcm.mem[5]);

    $display("TCM[6] = %08h",

DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen
_tcm.mem[6]);

    $display("TCM[7] = %08h",

DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.gen_tcm0.u_subsys_TCM_0.tcm_ram.u_ram_0.gen
_tcm.mem[7]);

  end



  initial begin

    // Wait until just before reset release
    #900;

    $display("");
    $display("============================================");
    $display(" RESET DEBUG - BEFORE RELEASE");
    $display("============================================");

    $display("[%0t ns] RESETN             = %b",
        $time, RESETN);

    $display("[%0t ns] toggle_hart_soft_reset = %b",
        $time,
```

```
DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.u_subsys_interconnect_0.u_subsys_regs.toggle_hart_soft_reset);

    $display("[%0t ns] hart_soft_reset_reg    = %b",
        $time,

DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.u_subsys_interconnect_0.u_subsys_regs.subsys_hart_soft_reset_reg);

    // Wait 200 ns after reset release
    #200;

    $display("");
    $display("================================================");
    $display(" RESET DEBUG - AFTER RELEASE");
    $display("================================================");

    $display("[%0t ns] RESETN              = %b",
        $time, RESETN);

    $display("[%0t ns] toggle_hart_soft_reset = %b",
        $time,

DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.u_subsys_interconnect_0.u_subsys_regs.toggle_hart_soft_reset);

    $display("[%0t ns] hart_soft_reset_reg    = %b",
        $time,

DUT.MIV_RV32_C1_0.MIV_RV32_C1_0.u_ipcore_0.u_subsys_interconnect_0.u_subsys_regs.subsys_hart_soft_reset_reg);
```

```
end



initial begin

   $display("");
   $display("================================================");
   $display(" Mi-V RV32 Firmware Simulation");
   $display("================================================");


   wait (LOCK == 1'b1);

   $display("");
   $display("[%0t ns] FCCC LOCK = 1", $time);


   #50000;

   $display("");
   $display("================================================");
   $display(" SIMULATION FINISHED");
   $display("================================================");

   $display("[%0t ns] Simulation finished", $time);

   $finish;

end
```

endmodule

Compilation result :

```
# ================================================
#  Loading Mi-V RV32 firmware into TCM
# ================================================
# Firmware loaded into Mi-V TCM
# TCM[0] = 00004117
# TCM[1] = 01010113
# TCM[2] = ff017113
# TCM[3] = a0012011
# TCM[4] = 123457b7
# TCM[5] = 80000737
# TCM[6] = 67878793
# TCM[7] = a001c31c
# ================================================
#  Mi-V RV32 Firmware Simulation
# ================================================
# [100000000 ns] RESETN released
VSIM 2> run -all
# [5755000000 ns] FCCC LOCK = 1
# [55755000000 ns] Simulation finished
# ** Note: $finish    : C:/Microchip/Libero_SoC_2026.1/krisemi_projects/firmware_application/stimulus/firmware_Sim.v(101)
#    Time: 55755010 ps  Iteration: 0  Instance: /testbench
```

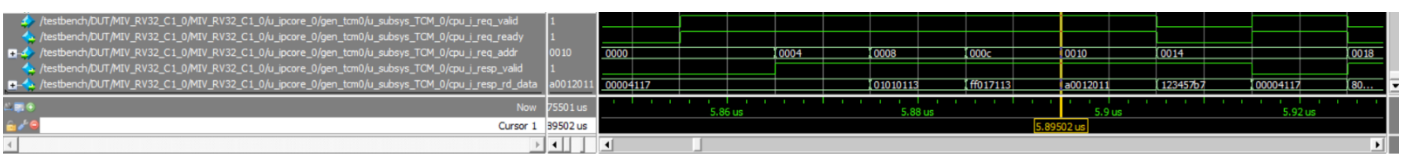
This was an important milestone.

It proves that:

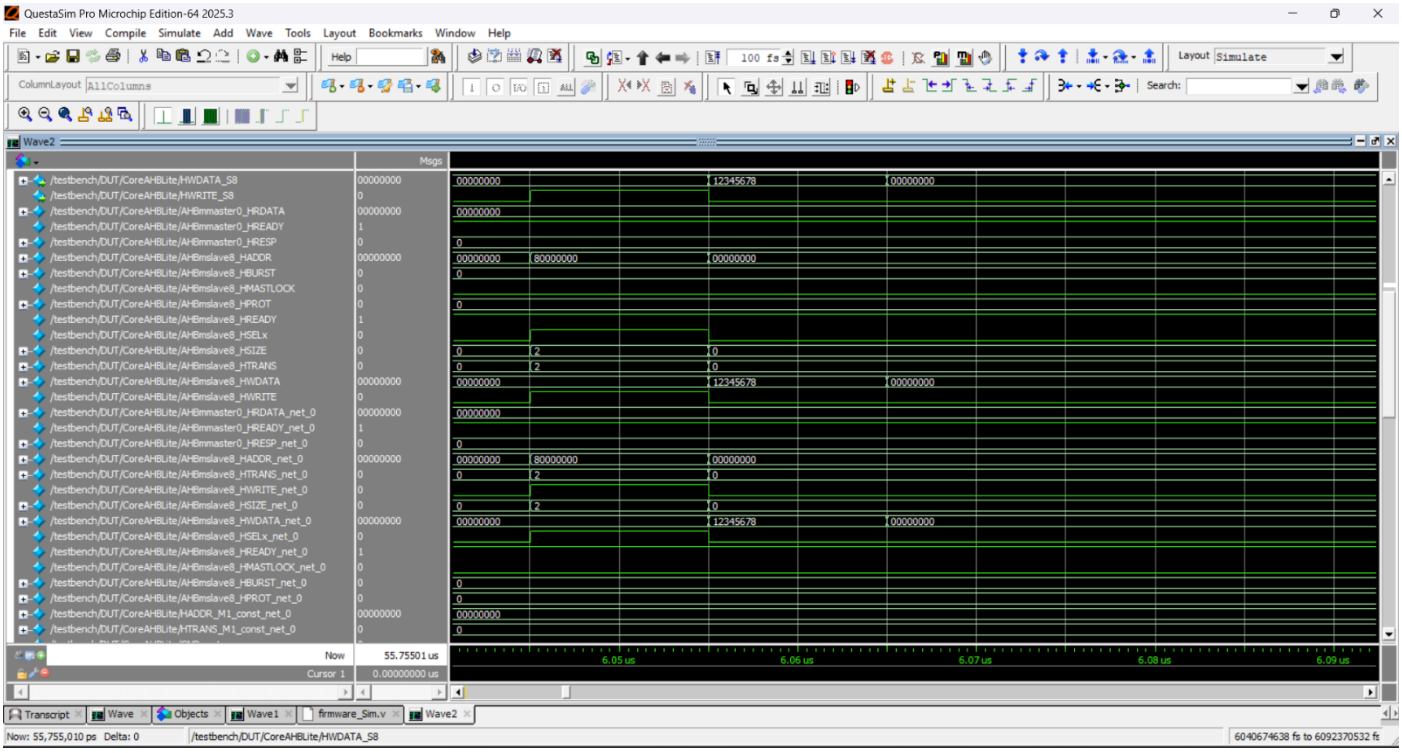
1. The .mem file is valid.
2. $readmemh successfully loaded it.
3. The hierarchical path reaches the **actual Mi-V TCM RAM**.
4. The expected firmware words are present in the RAM before execution.

Mi-V RV32 Instruction Fetch Verification:

After correcting the reset release sequencing, the Mi-V RV32 successfully initiated multiple instruction fetch transactions from the integrated TCM. The instruction request address advanced sequentially from 0x0000 through 0x0014, with valid instruction responses observed from the TCM.

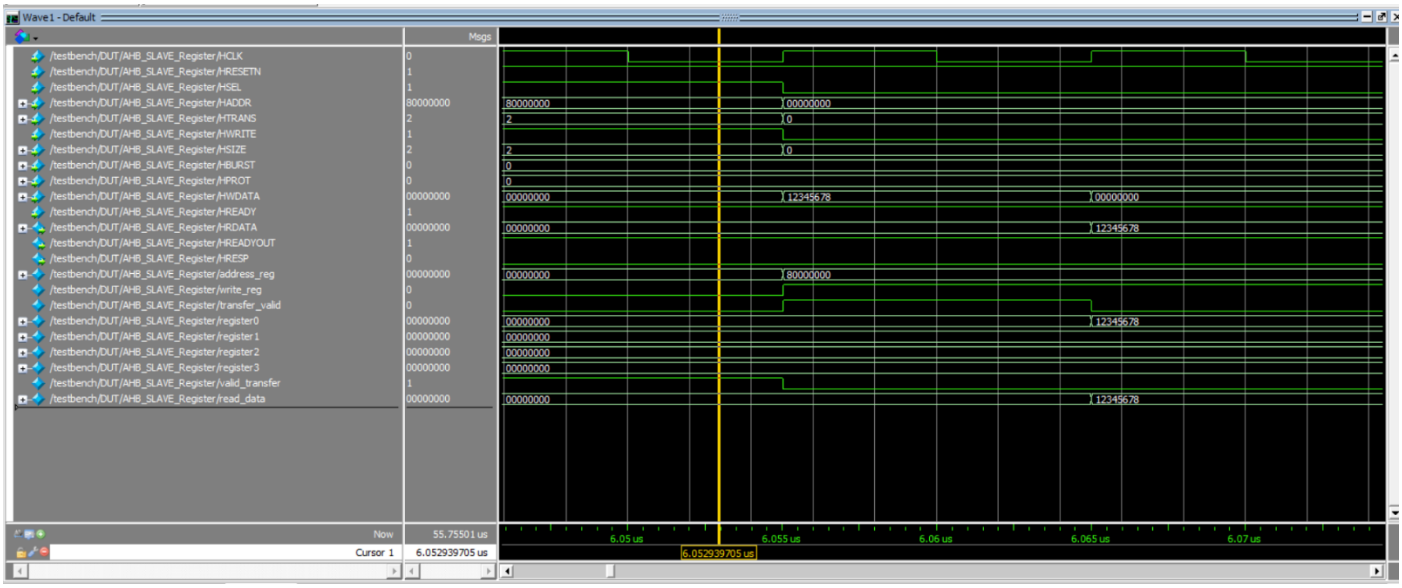


CoreABH_Lite transaction



CoreAHBLite signal	**Observed**
AHBmslave8_HADDR	0x80000000
AHBmslave8_HWDATA	0x12345678
AHBmslave8_HWRITE	1
AHBmslave8_HTRANS	2 (NONSEQ)
AHBmslave8_HSIZE	2
AHBmslave8_HSELX	1
AHBmslave8_HREADY	1
AHBmslave8_HRESP	0

AHB_Slave transaction :



Signal	**Observed**
HSEL	1
HADDR	0x80000000
HTRANS	2 → NONSEQ
HWRITE	1
HSIZE	2
HBURST	0
HWDATA	0x12345678
HREADY	1
HRESP	0

address_reg = 0x80000000

transfer_valid = 1

and subsequently: register0 = 0x12345678

So the data has actually been accepted and stored by the slave.

Check Sheet:

Current Verification Status

The current status can be summarized as:

Verification Item	**Status**
Mi-V integrated into SmartDesign	PASS
MSS removed from active architecture	PASS
CoreAHBLite connection	PASS
AHB Slave connection	PASS
Mi-V TCM enabled	PASS
Reset vector configured	PASS
SoftConsole RISC-V project	PASS
Linker configured for TCM	PASS
Firmware compilation	PASS
HEX generation	PASS
HEX → MEM conversion	PASS
$readmemh TCM loading	PASS
TCM contents verified	PASS
FCCC clock lock	PASS
First instruction fetch	PASS
Second instruction fetch	PASS
Firmware reaches main()	CONFIRMED
AHB write 0x12345678	CONFIRMED

Current Project Conclusion

The **Mi-V RISC-V based MSS/FPGA system has been successfully integrated and functionally verified at the simulation level**. The Mi-V processor is able to fetch instructions from the configured TCM, confirming that the firmware image is correctly loaded and accessed by the processor. The observed instruction data from the IFU matches the corresponding TCM contents.

The **AHB communication path has also been verified**. A valid AHB write transaction with address 0x80000000 and data 0x12345678 was observed passing through the **CoreAHBLite** interconnect to the custom AHB_SLAVE. The slave successfully captured and stored the data in its register, and the same value was subsequently observed during readback.

Debugging Learnings

During the development and simulation of the MSS_FPGA design, several important debugging lessons were identified. These helped in understanding the interaction between the Mi-V RISC-V processor, TCM, clock/reset infrastructure, CoreAHBLite, and the custom AHB slave.

1. Reset and Clock Sequencing

A major issue was identified where the external clock was running, but the internal Mi-V clock generated through the FCCC/CCC was not active until the clock lock was established. Releasing RESETN before the internal clock became active resulted in internal registers remaining in an unknown (X) state. The reset sequence was therefore changed to wait for LOCK = 1 before releasing reset.

Learning: Reset release must be synchronized with the availability of the internally generated processor clock.

2. Understanding Simulation Hierarchy

Several debugging issues were caused by confusion between the design hierarchy, simulation hierarchy, and generated IP hierarchy. Checking the exact hierarchy before probing internal signals was found to be important.

Learning: Always verify the actual simulation hierarchy before adding or examining internal signals.

3. TCM and Instruction Fetch Debugging

The firmware was loaded directly into the inferred TCM using $readmemh. Instead of assuming that the processor was executing the firmware, the instruction-fetch interface was traced through the IFU and TCM.

For example:

TCM[7] = A001C31C

IFU response data = A001C31C

This provided direct evidence that the Mi-V processor was receiving instructions from the loaded TCM contents.

Learning: In processor debugging, verify the complete path:

Firmware → TCM → IFU request → TCM response → CPU

rather than relying only on the CPU PC.

4. examine vs Waveform Debugging

It was observed that Questa's examine command reports the signal value at the **current simulation time**, not its historical value. Therefore, using examine alone can be misleading when trying to understand what happened earlier in the simulation.

Learning: Use the waveform for time-dependent debugging and use examine mainly for checking the current signal state.

5. AHB Transaction-Level Debugging

The AHB path was debugged by following a transaction through the system rather than checking individual modules independently.

The transaction:

HADDR = 0x80000000

HWRITE = 1

HWDATA = 0x12345678

was observed at the CoreAHBLite and AHB slave interfaces.

Learning: For bus-based systems, trace the same transaction from **master → interconnect → slave → return path**.

6. CoreAHBLite Address Decoding

The CoreAHBLite waveform showed HSELX = 1 for the selected slave while the address and write data were correctly propagated.

This confirmed that the address decoding and slave selection were functioning as expected.

Learning: For interconnect debugging, HSEL, HADDR, HTRANS, HWRITE, and HWDATA are key signals for establishing whether a transaction reaches the intended peripheral.

7. AHB Slave Register Verification

The custom AHB slave was verified beyond simply observing bus activity. The written value:

0x12345678

was stored in register0 and subsequently returned during readback.

Learning: A peripheral should be considered functionally verified only after confirming both **write → storage** and **read → readback** behavior.

8. Avoiding Premature Root-Cause Conclusions

The simulation produced several port-width warnings, including 14-bit/32-bit address mismatches. These warnings were recorded, but they were not immediately treated as the cause of the observed behavior because the relevant TCM and AHB transactions were functioning correctly.

Learning: A simulation warning should be investigated, but its causal relationship to a failure should be established through evidence rather than assumption.